Notes on Combinatorial and Probabilistic Algorithms

Riccardo Lo Iacono

August 11, 2026

Contents.

– 1 – The analysis of Algorithms.

As a good computer scientist knows, not all algorithms are the same nor are the resouces these use; it is for these reasons that we need ways to analyze algorithms. In what follows, we first recall what asymptotic notation is, and then introduce the notion of *amortized analysis*.

– 1.1 – Asymptotic notation.

That idea behind asymptotic notation is that of analyzing algorithms, with respect to a given resource, more often then not being time, without caring about constants.

We recall the existence of three major notation: the *omega* notation (Ω), the *theta* notation (Θ) and the big-o notation ($\mathcal{O}$). Let $g(n)$ and $f(n)$ be two functions, then we define each notation as follow:

$$\begin{aligned}
\Omega(f(n)) =& \{g(n) \mid \exists c > 0, n_0 \geq 0 \text{ s.t. } g(n) \geq cf(n), \forall n \geq n_0\} \\
\Theta(f(n)) =& \{g(n) \mid \exists c_1, c_2 > 0, n_0 \geq 0 \text{ s.t. } c_1 f(n) \leq g(n) \leq c_2 f(n), \forall n \geq n_0\} \\
\mathcal{O}(f(n)) =& \{g(n) \mid \exists c, n_0 \geq 0 \text{ s.t. } g(n) \leq cf(n), \forall n \geq n_0\}
\end{aligned}$$

Although we give the definition in set notation, we write $g(n) = \mathcal{O}(f(n))$ to mean $g(n) \in \mathcal{O}(f(n))$; we do similarly for the other notations. Additionally, unless stated otherwise, assume we are interested in the time complexity of the algorithm.

– 1.2 – Amortized analysis.

Let us consider the following: assume we are given a data structure, and we are asked to compute a sequence of m operations on it; what is the overall complexity of all m operations, i.e, what is the time needed to compute all m operations.

It follows easily that, applying a worst case analysis, we need $m \cdot T_{MAX}(n)$, where $T_{MAX}(n)$ is the time complexity of the most expesive operation. For instance, under the worst case assumption, a sequence of m deletion from a tree has a complexity of $\mathcal{O}(m \cdot n)$.

We now introduce the notion of *amortized analysis*, and show how, over a sequence of operations, the real time complexity of an algorithm differs from the worst case one.

In its essence, amortized analysis does nothing more than averaging the complexity of all operations. That is, if $T_1(n)$ is the complexity of the first operation, $T_2(n)$ the one for the second operation, ..., $T_m(n)$ the complexity

of the m-th operation, then the amortized time is given as

$$T_{amortized}(n) = \frac{1}{n}\sum_{i=1}^{m} T_i(n).$$

In other words, amortized time represents the time needed for each of the m operations.

The next section describes a simple ways to perform amortized analysis.

– 1.2.1 – Accounting method.

The core idea of the method follows that used by banks. In more details: to each operation we assign a cost, which represents its *amortized cost*. When the amortized cost of an operation exeeds its real cost, we are left with some credit that we can use to help pay other operations.

Example: Consider we need an algorithm to increment a n bits counter by an amount m. From a worst case analysis perspective, the algorithm needs a quadratic time, as the n-th bit may need to be flipped m times. It comes with no surprise that such a cost is well above the real one, as we are about to see.

Let us proceed this way: for each bit flip, assume that we are asked to pay a coin; then we proceed by paying two coins when flipping a bit to 1. The cost of the whole algorithm is easily seen to be given by the number of total bit flipped. But the following can be observed: the first increment, leaves us with a coin of credit, the second one does the same, etc. Thus, each operation requires a constant amount; that is, the amortized cost of the whole algorithm is $\mathcal{O}(n)$.

Remark: Other methods to perform amortized analysis exist, and their use varies with the algorithm we need to analyze. Though, all follow the same core idea: we overpay for some operations and use the credit thus produced to pay for others.

– 2 – The dictionary problem.

Now, we focus our attention to one of the most common problems in computer science, namely, the *dictionary problem* which can be roughly defined as follow: given a set of items S, define a data structure such that the following three operations are performed efficiently.

Search(S, x)
Insert(S, x)
Delete(S, x)

The following assumption on operations cost is taken: accessing or deleting an item costs i while an insertion costs $i + 1$, where i is the length of the list preceeding the item.

We begin our discussion on the problem with the following trivial remark: if the elements have some kind of total order relation, then trees provide the best approach; if not, the best possible solution is given by lists.

Here we discuss two type of solutions, namely, self-adjusting lists (for the case in which no order relation exists) and self-adjusting trees (for the other case). Specifically, for self-adjusting trees we discuss, in order, red-black trees (Section 2.2.1) and splay trees (Section 2.2.2).

– 2.1 – Self-adjusting lists.

Our discussion on self-adjusting lists will proceed as follow: first, we introduce the definition of self-adjusting list; then, we focus our attention on some of the most common rules used to keep a list organized.

Conceptually, self-adjusting lists are trivial: these are lists in which, after accessing or inserting an item, some update procedure is called to keep the list organized. These procedure are defined according to some rules, among which those that stands out are

- *Frequency Count (FC)*: we mantain an array that stores at each position i, the count of how many times item i has been accessed or added;
- *Move-To-Front (MTF):* each time an item is added, it is moved to the head of the list, while each accessed item is moved slighly closer to the root;
- *Transpose (T):* accessing or inserting an item, implies an exchange of the added/accessed item with the one on its left.

Let $\mathbb{E}_A[p]$ be the expected cost to access an item using algorithm A, where $p = \{p_1, p_2, \ldots, p_n\}$ is the probability distribution associated to the sequence of operations. Assume DP to be an algorithm that orders the elements according

to p. By the law of large numbers it follows $\mathbb{E}_{FC}[p]/\mathbb{E}_{DP}[p] = 1$. Additionally, one could prove that $\mathbb{E}_{MTF}[p] \leq 2\mathbb{E}_{DP}[p]$, and as proved in [4] $\mathbb{E}_{T}[p] \leq \mathbb{E}_{MTF}[p]$. A similar result can be easily derived for insertions and deletions. From the above, it follows that, at least in theory, both FC and transpose out perform MTF. As we are about to show, theory and practice not always coincide. What we want to show is that, although in theory FC and transpose are better then MTF, in practical applications MTF performs better.

For any algorithm A, let us denote with $C_A(s)$ the total cost of all operations, by $F_A(s)$ the number of exchanges that move an item to the head of the list, and by $P_A(s)$ any other exchange. The following holds.

Theorem 2.1 ([5, Theorem 1.]): *For any Algorithm A and any sequence of operations s starting with the empty set*

$$C_{MTF}(s) \leq C_A(s) + P_A(s) - F_A(s) - m.$$

Theorem 2.1 and its generalized version (see [5, Theorem 2.]), show that MTF differs by any other algorithm by a most a factor of 2. No analogous result holds for FC and transpose.

– 2.2 – Self-adjusting trees.

Our discussion on self-adjusting trees will proceed as follow: first, we introduce Red-Black Trees (RB) and we overview the operations on these, then, we describe splay trees.

– 2.2.1 – Red-Black trees.

A special variant of Binary Search Trees (BST), RB trees allow to implement the operations of Search(S, x), Insert(T, x) and Delete(T, x) in $\mathcal{O}(\log n)$ time.

Formally speaking, RB trees are binary search trees satisfying the following properties:

1. Each internal node is either red or black.
2. The root is black.
3. Each leaf is black.
4. Each red node has only black children.
5. Any simple path from a node to a descendant leaf, contains the same number of black nodes.

Property 5 implicitly defines a measure to which, for a given node x, we refer to as the black height $\texttt{bh}(x)$. That is, $\texttt{bh}(x)$ is the number of black nodes

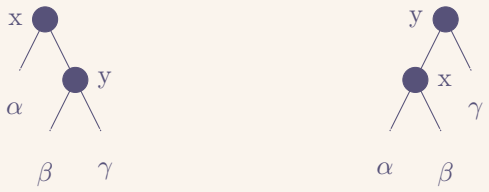
in the path from x to any of its descendant leaf. The following result easily follows.

Lemma 2.1: *If T is a Red-Black tree with n nodes, then T has height at most* $2\log(n+1)$.

Proof. Follows immediately by a simple induction on n. ■

Rotations.

As we shall see, insertion and deletion on a RB tree are everything but simple. We shall in fact see that either of the two operations may cause one or more properties to fail. To fix this issues specific procedures are used to restore each property; in doing so some local pointer manipulations are made, which here we refer to as rotations. We describe left rotations, the other being symmetrical.



(a) **(b)**

Figure 1: An example of left (resp. right) rotation.

Consider the case shown in Figure **(a)**. To obtain the tree in Figure **(b)** we proceed as follow: we let y's left child to be x's new right child, and we make x be the new y's left child.

Insertion.

In our discussion on RB insertion we assume the following: for any node x, $x.r$ denotes the right child of x, likewise $x.l$ denoets the left child, $x.p$ denotes x's parent, and $x.c$ is x's color. Additionally, for any inserted node x, $x.l$ and $x.r$ are set to nil, which we assume to be black, and $x.c$ is red. We refer to the inserted node with z.

We now discuss RB insertion focusing on which of the RB properties may fail after insertion, and how these are fixed.

First, observe that after insertion every node is either red or black, and thus Property 1 holds, similarly Property 3 holds as the newly inserted node has $z.l$ and $z.r$ set to nil, and so does Property 5 since no new black node has been added. On the other hand, Property 2 may fail if the inserted node replaces the root, while Property 4 fails when $z.p$ is red.

With respect to Procedure RB-Insert, which shows the pseudo code to implement RB insertion, we show how RB−Insert−Fixup restores each of the RB

Procedure RB-Insert(T, z)

```
y = nil ;
x = T.root
while x ≠ nil do
    y = x
    if z.key < x.key then
        x = x.left
    else
        x = x.right
end
z.p = y
if y == nil then
    T.root = z
else if z.key < y.key then
    y.left = z
else
    y.right = z
z.left = nil
z.right = nil
z.color = "Red"
RB-Insert-Fixup(T, z)
```

properties.

Case 1: z's uncle y is red. Since both $z.p$ and y are red, and $z.p.p$ exists and is black, we simply "push" the redness upward and call RB−Insert−Fixup on $z.p.p$. That is, we recolor $z.p$ and y black, while recoloring $z.p.p$ red.

Case 2: z's uncle y is black and z is a right child. To be considered a subcase of Case 3, we simply call Left−Rotate on z and fall into Case 3.

Case 3: z's uncle y is black and z is a left child . Either it is accessed directly or through Case 2, we simply call Right−Rotate on z and apply some recoloring.

The only property that may still fail is Property 2, which a simple recoloring at the end of RB−Insert−Fixup solves.

It shall be evident that upon RB−Insert−Fixup termination, all RB properties are restored.

Procedure RB-Insert-Fixup(T, z)

```
while z.p.color == "Red" do
    if z.p == z.p.p.left then
        y = z.p.p.right ;
        if y.color == "Red" then
            z.p.color = "Black"
            y.color = "Black"
            z.p.p.color = "Red"
            z = z.p.p
        else if z == z.p.right then
                z = z.p
                Rotate-left(T, z)
            z.p.color = "Black"
            z.p.p.color = "Red"
            Rotate-right(T, z.p.p)
    else
        /* Likewise with "right" and "left" swapped.       */
end
T.root.color = "Black"
```

Deletion.

Our discussion on RB deletion will assume the following: for any node y, $x.r$ denotes x's right child, similarly $x.l$ denotes the left child, with $x.p$ we indicate

x's parent and with $x.c, x$'s color. Additionally, we denote with z the node to remove, with y the node the replaces the deleted node and we let it always point to z, and with x the node filling y's position after this has been moved.

As for insertion, we mainly focus on the properties that may fail upon deletion, and explain how these are fixed.

First, if y was the root and a red child replaces it, Property 2 fails. Second, if x and $x.p$ are both red, then Property 4 fails. Third, moving y may cause Property 5 to fail. We can correct the violation of Property 5 by saying that node x, now occupying y's original position, has an "extra" black. That is, if we add 1 to the count of black nodes on any simple path that contains x, then under this interpretation, property 5 holds. When we remove or move the black node y, we "push" its blackness onto node x. The problem is that now node x is neither red nor black, thereby violating property 1.

Procedure RB-delete(T, z)

```
y = z
y − starting − color = y.color
if z.left == nil then
    x = z.right
    RB − transplant(T, z, z.right)
else if z.right == nil then
    x = z.left
    RB − transplant(T, z, z.left)
else
    y = Tree − Min(z.right)
    x = y.right
    if y.p == z then
        x.p = z
    else
        RB − transplant(T, y, y.right)
        y.right = z.right
        y.right.p = y
    RB − transplant(T, z, y)
    y.left = z.left
    y.left.p = y
    y.color = z.color
if y − starting − color == "Black" then
    RB − delete − fixup(T, x)
```

With respect to Procedure RB-delete, which shows the pseudo code to implement RB deletion, we show how RB−Delete−Fixup restores each of the RB

properties.

Case 1: z's sibling w is red. We know that w's must be black, therefore we can exchange the colors of w and $x.p$ and apply a left rotation on $x.p$.

Case 2: z's sibling w is black, and so are its children. Since both w's children are black, and so are z and w, we can simply pulling away a black from both, leaving z singly black and w red.

Case 3: z's sibling w is black, w.l is red and w.r is black. We can switch the colors of w and its left child $w.l$ and then perform a right rotation on w.

Case 4: z's sibling w is black, and w.r is red. By making some color changes and performing a left rotation on $x.p$, we can remove the extra black on x, making it singly black.

Upon termination Property 2 may still fail, which is fixed at the end of RB–Delete–Fixup.

Procedure RB-delete-fixup(T, x)

```
while x ≠ T.root and x.color ≠ "Black" do
    if x == x.p.left then
        w = x.p.right
        if w.color == "Red" then
            w.color = "Black"
            x.p.color = "Red"
            Rotate − left(T, x.p)
        if w.left.color == "Black" and w.right.color == "Black"
         then
            w.color = "Red"
            x = x.p
        else if w.right.color == "Red" then
            w.left.color = "Black"
            w.color = "Red"
            Rotate − right(T, w)
            w = x.p.left
        w.color = x.p.color
        x.p.color = "Black"
        Rotate − left(T, x.p)
        x = T.root
    else
        ( same as above with "right" and "left" exchanged )
end
x.color = "Black"
```

– 2.2.2 – Splay trees.

Introduced in [6] by Sleator and Tarjan, splay trees are a variant of BSTs that are proved to be, in an amortized sense, optimal. In their paper the authors consider the following situation: assume we are asked to perform a sequence of access operation on a set of items selected from a totally ordered universe. The symmetry with the dictionary problem is obvious.

The key observation of the authors is that, when accessing an item, it is very likely that said item will be accessed soon, meaning that moving it toward the root will reduce the time required for the subsequent aceess operation. The idea, originally proposed in [1] by Bitner, is improved in [6] to avoid the $\mathcal{O}(n)$ time cost on arbitrary long sequences of operations. To achive so, they propose a new heuristic procedure called splaying.

Here we discuss splaying, how this is used to implement the dictionary operations, and report some theoretical results regarding splay trees. See [6, 7] for more details.

Let x be the least recently accessed node, and let $x.p$ denote x's parent; we can divide the splaying procedure in the following three cases:

Case 1 (zig): x.p is the root. Then, we simply left (resp. right) rotate on x, depending if it was a right (resp. left) child of $x.p$.

Case 2 (zig-zig): both x and x.p are left (resp. right) children. We first right (resp. left) rotate on $x.p$, making $x.p$ a right child of x, then we right rotate on x.

Case 3 (zig-zag): x is a right child and x.p is a left child (or vice versa). First, we left rotate on x, making $x.p$ be a left child of x. Then, we right rotate on x and make the new parent a right child.

A visual representation of each case is shown in Figure 2. As for for RB trees, rotations are local operation on nodes, therefore the order of the elements is left unchanged.

The following analysis of the splaying procedure is based on the assumption that each to node x is associated an arbitrary weight $w(x)$. Additionally, we use the following notation: $s(x)$ denotes the size of x, i.e., the sum of the weight in the subtree rooted at x, and by $r(x)$ the rank of x, defined as the logarithm of $s(x)$. For a tree T, we use $r(T)$ to denote the rank of the root of T.

Theorem ([6, Lemma 1]): *The amortized cost for splaying a tree T at a node x is at most $3(r(T) - r(x)) + 1$.*

Proof. If no rotations occur, the bound follow immediately. Hence, suppose at least a rotation is performed. Let s and s', r and r' be the weight and the rank before and after the rotation, respectively. We state that the amortized

cost for the step is $3(r'(x) - r(x)) + 1$ for Case 1, and $3(r'(x) - r(x))$ for Case 2 and 3. Let $y = x.p$ and $z = y.p$, we have:

Case 1. One rotation occurs; the amortized time is therefore

$$\begin{aligned} &1 + r'(y) - r(y) + r'(x) - r(x) && \text{since only } y \text{ and } x \text{ change rank} \\ &\leq 1 + r'(x) - r(x) && \text{since } r'(y) \leq r(x) \\ &\leq 1 + 3(r'(x) - r(x)) && \text{since } r'(x) \geq r(x) \end{aligned}$$

Case 2. Two rotations occur; the amortized time is

$$\begin{aligned} &2 + r'(z) - r(z) + r'(y) \\ &\quad - r(y) + r'(x) - r(x) && \text{since only } z, y \text{ and } x \text{ change rank} \\ &\leq 2 + r'(z) + r'(y) - r(y) + r(x) && \text{since } r(z) = r'(x) \\ &\leq 2 + r'(z) + r'(x) - 2r(x) && \text{since } r'(x) \leq r'(y) \text{ and } r(y) \geq r(x) \end{aligned}$$

Case 3. Two rotations are done, so the amortized time is

$$\begin{aligned} &2 + r'(z) - r(z) + r'(y) \\ &\quad - r(y) + r'(x) - r(x) \\ &\leq 2 + r'(z) + r'(y) - 2r(x) && \text{since } r'(x) = r(z) \text{ and } r(x) \leq r(y) \end{aligned}$$

The theorem follows by considering the maximum of the three cases. Hence, splaying at tree takes at most $3(r(T) - r(x)) + 1$. ■

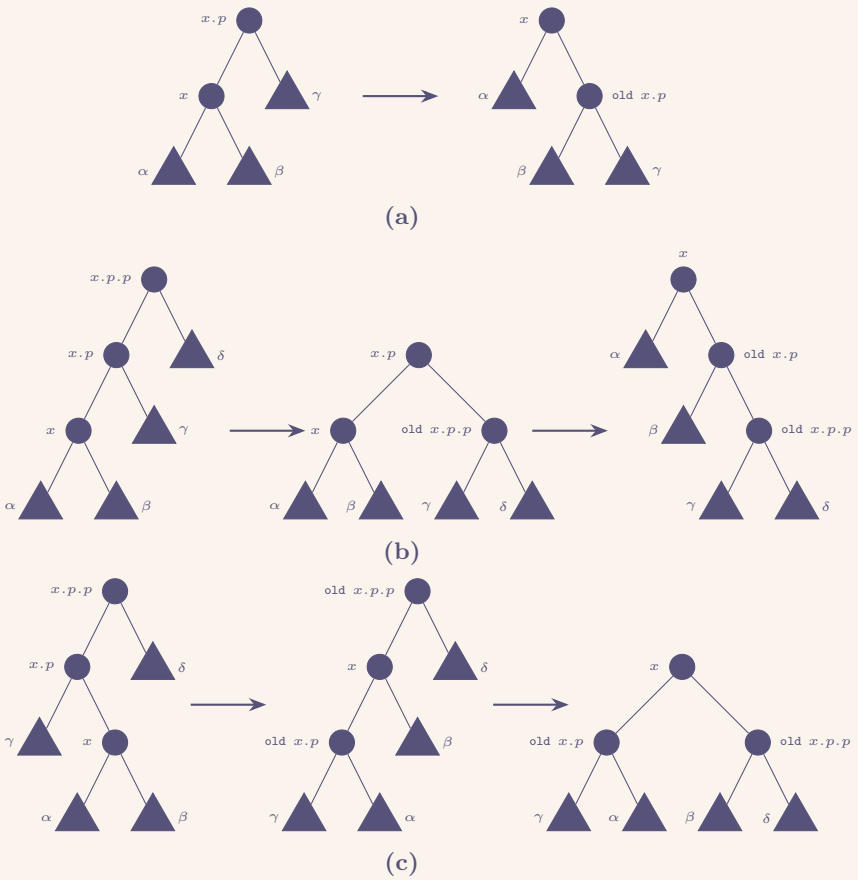


Figure 2: Example of splaying steps. Circles denotes internal nodes, whilst triangles denote arbitrary subtrees. **(a)** shows the case in which $x.p$ is the root. As stated, a simple rotation to the right will suffice. **(b)** shows the case in which both x and $x.p$ are left children; a left rotation on $x.p$ first, and a right one on x suffice. **(c)** shows the case in which x is a right child, and $x.p$ a left child; we rotate on x twice.

– 3 – Hashing.

We have seen how trees and lists can be used to implent dictionaries. Now we discuss another way to implent dictionaries when the set of possible keys K is much greater than the number of key that can be stored. In other words, when $|K| \gg |T|$, where T is the structure used to store the keys.

In this section, we first recall the idea behind hashing, we then introduce the notion of universal hash function, and show how, under some reasonable assumptions, we can achive a $\mathcal{O}(1)$ time for any of the dictionary operations.

– 3.1 – What is hashing?.

At least in principle, hashing is very simple: we use function $h : K \to \{0, 1, m-1\}$, where $m = |T|$, to map each of the keys in k to one of the slots in T.

It comes with no surprise that any two keys $k, l \in K$ can be such that $h(k) = h(l)$. If this happens, we say that a collision has occured. Therefore we need a way to handle collisions. The literature provides several solutions, the simplest of which is *chaining*. Under chaining each slot i points to a list L_i, which in turns stores all the keys in K that maps to i. In other words, for each i

$$L_i = \{k \in K \text{ s.t. } h(k) = i\}.$$

From what just discussed, it follows that we are interested in finding hash functions to minimise collisions. A class of such functions is discussed in the following section.

– 3.2 – Universal hashing.

Consider the case in which the keys are choosed in such a way that, after the hashing, the average retrival time is $\Theta(n)$, a situation that can happen with a non-zero probability. The use of fixed hash functions may lead to such situations. A simple solution, is that of choosing the hash function at random from a family of well-designed hash functions.

Here, we first show why this kind of solution works, then, we show how to one of such families is defined.

Let $\mathcal{H}$ be a finite collection of hash functions. We say that $\mathcal{H}$ is universal if, for any pair of keys $k, l \in K$, the number of hash function such that $h(k) = h(l)$ is no more than $|\mathcal{H}|/m$. In other words, $\mathcal{H}$ is universal if the probability of a collision is less than $1/m$.

The following theorem shows that universal has functions give a good-average case.

Theorem 3.1: *Let h be a function that has been used to hash n keys into the table T of size m. If key k is not in the table, then the expected lenght of the*

list k hashes to is at most $n/m = \alpha$. Viceversa, if key k is in the table, the expected lenght of the list storing k is at most $1 + \alpha$.

Proof. Let X_{kl} be the indicator random variable $I\{h(k) = h(l)\}$. By definition of universal universal collection of hash functions, $\Pr(h(k) = h(l)) \leq 1/m$. By a simple result in probability theory, we know $\mathbb{E}[X_{kl}] \leq 1/m$.

Let Y_k denote the number of keys $l \neq k$ that maps to the same slot as k. We have

$$\begin{aligned}\mathbb{E}[Y_k] &= \mathbb{E}\left[\sum_{l \in K, l \neq k}\right] \\ &= \sum_{l \in K, l \neq k} \mathbb{E}[X_{kl}] \\ &\leq \sum_{l \in K, l \neq k} \frac{1}{m}\end{aligned}$$

If $n_{h(k)}$ is the lenght of the list k hashes to, then

- if $k \notin T$, then $\mathbb{E}\left[n_{h(k)}\right] = \mathbb{E}[Y_k]$ and $|\{l \mid l \in T \wedge l \neq k\}| = n$. Therefore, $\mathbb{E}\left[n_{h(k)}\right] \leq n/m = \alpha$.
- if $k \in T$, then $\mathbb{E}\left[n_{h(k)}\right] = \mathbb{E}[Y_k] + 1$ and $|\{l \mid l \in T \wedge l \neq k\}| = n - 1$. Hence, $\mathbb{E}\left[n_{h(k)}\right] \leq (n-1)/m + 1 = 1 + \alpha - 1/m \leq 1 + \alpha$

■

– 3.2.1 – Designing a universal class of hash functions.

We start by choosing a prime number p such that all possible key fall in the range $(0, p-1)$. Consider the group $\mathbb{Z}_p$ and $\mathbb{Z}_p^*$. Since p is prime we can solve equations modulo p.

For any $a \in \mathbb{Z}_p^*$ and $b \in \mathbb{Z}_p$, let h_{ab} be the function defined as follow:

$$h_{ab}(k) = ((ak + b) \mod p) \mod m$$

and let $\mathcal{H}_{pm}$ be the class of all such functions.

Theorem 3.2: *The family of hash functions $\mathcal{H}_{pm}$ is universal.*

Proof. For any two distinct keys $k, l \in \mathbb{Z}_p$, and an hash function h_{ab} we have

$$\begin{aligned}h_{ab}(k) = r &= ((ak + b) \mod p) \mod m \\ h_{ab}(l) = s &= ((al + b) \mod p) \mod m\end{aligned}$$

Some simple algebra shows that $r \neq s$. Hence, at the modulo p level no collision can occur.

Therefore, the probability that two keys hash to the same value coicide with the probability that $r \equiv s \mod m$. Now, if r and s are chosen at random

from $\mathbb{Z}_p$, for a fixed value of r, the number of values for s such that $s \neq p$ and $s \equiv p \mod m$ is at most

$$\begin{aligned}\lceil p/m \rceil - 1 &\leq ((p + m - 1)/m) - 1 \\ &= (p - 1)/m.\end{aligned}$$

Meaning that the probability that $r \equiv s \mod m$ is at most $\frac{(p-1)/m}{(p-1)} = 1/m$. Hence, $\Pr(h_{ab}(k) = h_{ab}(l)) \leq 1/m$, and thus $\mathcal{H}_{pm}$ is universal.

– 3.3 – Perfect hashing.

So far we have considered the case in which the set of keys K was dynamic. We now turn our attention to the case in which K is static, and we show that in this case a constant time lookup worst case is achiveable.

The scheme we are about to discuss is often refferred to as *perfect hashing*. The idea is the following: we use a two level schema in which, by means of a hash function h randomly choosed from a family of universal hash functions, each of the n keys is mapped to one of the m slots of the table. Each slot j stores a secondary hash table of size $m_j = n_j^2$ to which is associate a hash function h_j, where n_j is the number of keys mapping to j.

Assume that the first level hash function comes from $\mathcal{H}_{pm}$, while each of the secondary level hash functions come from $\mathcal{H}_{pm_j}$. We show that, choosing h_j carefully we can garantee that no collision will occur. Additionally, we show that the overall space required by both the primary table and the secondary one is amounts to $\mathcal{O}(n)$. The following theorems and corollary will do the work.

Theorem: *Let h be a universal hash function, and let $m = n^2$ be the size of a hash table used to store a static set of keys K of size n. Then, the probability of a collision occuring is les than $1/2$.*

Proof. Since h is a universal hash function, any two keys collide with probability $1/m$. The number of possible pair that may collide is $\binom{n}{2}$. If X is a random variable that counts the number of collisions, then

$$\begin{aligned}\mathbb{E}[X] &= \binom{n}{2}\frac{1}{m} \\ &= \binom{n}{2}\frac{1}{n^2} && \text{since } m = n^2 \\ &= \frac{n^2 - n}{2}\frac{1}{n^2} \\ &< \frac{1}{2} && \text{as } n \to \infty\end{aligned}$$

■

Theorem: *Let h be a hash function randomly chosen from a family of universal hash functions, and assume we store n keys in a table of size $m = n$. Then, it holds*

$$\mathbb{E}\left[\sum_{i=0}^{m-1} n_j^2\right] < 2,$$

n_j being the number of keys hashing to j.

Proof. By a simple induction we have that the following relation holds for every non-negative integer:

$$k^2 = k + 2\binom{k}{2}.$$

It follows

$$\begin{aligned}
\mathbb{E}\left[\sum_{i=0}^{m-1} n_j^2\right] &= \mathbb{E}\left[\sum_{i=0}^{m-1} n_j\right] + 2\mathbb{E}\left[\sum_{i=0}^{m-1} \binom{n_j}{2}\right] && \text{by the linearity of expectation} \\
&= \mathbb{E}\left[n\right] + 2\mathbb{E}\left[\sum_{i=0}^{m-1} \binom{n_j}{2}\right] \\
&= n + 2\mathbb{E}\left[\sum_{i=0}^{m-1} \binom{n_j}{2}\right] && \text{since } n \text{ is not a random variable}
\end{aligned}$$

Now, recall that h is a universal hash function, hence

$$\begin{aligned}
\mathbb{E}\left[\sum_{i=0}^{m-1} \binom{n_j}{2}\right] &\leq \binom{n}{m}\frac{n}{m} \\
&= \frac{n(n-1)}{2m} \\
&= \frac{n-1}{2}
\end{aligned}$$

since $m = n$. Therefore,

$$\mathbb{E}\left[\sum_{i=0}^{m-1} n_j^2\right] \leq n + 2\frac{n-1}{2} < 2n$$

■

Corollary: *Let h be a hashing function randomly chosen from a family of universal hash functions, and assume we store n keys in an hash table of size $m = n$, and we set the size of each secondary hash table to $m_j = n_j^2$. Then, under a perfect hashing schema, the amount of space required is no more than $2n$.*

Proof. Follows immediately from the previous theorem. ■

– 3.4 – Bloom's filter.

We now turn our attention to the following problem: given a set M of items, how can we efficiently store the items in M, while also being able to immediately tell if an item $i \in M$? That is, is it possible to determine if $i \notin M$ by simply looking at the data structure?

By now, it should be clear that such a problem is easily solved through hashing, with the only issue concerning space. In fact, if the space available for the hast table is much less than that actually needed, the usual hashing approach is infeasable.

Here, a slightly different approach is discussed. In more details, we will overview the solution proposed in [2] by Bloom, and refer the interested reader to the paper for major details. We focus on what in [2] is referred to as "Method 2"; commonly known as Bloom's filter in the literature.

The idea is the following: we represent our hash table as a collection of N individually addressable bits, initially set to zero. If M is the set of items to store, then for each item $k \in M$ we identify a group of bits $b_1, b_2, \ldots, b_d$, by means of some hash procedure, and set these to one. Similarly, if i is some item we wish to check belongs to the set, then we compute the hash for i and check if all the bits (in the hash) are set to one.

Remark: It should be clear that, if for a given item k, all the bits in its hashing are set to one, then there's no guarantee that k belongs to the set. On the contrary, if any of the bits in the hashing is set to zero, then we are sure k doesn't belong to the set.

Stated differently, we allow some items to be falsely recognized as belonging to the set, to immediately discard those items that do not. We refer to these falsely recognized items as the *allowable fraction of error*.

We now proceed with the analysis of Bloom's filter. Let ϕ be the expected proportion of bits still set to zero after n items have been stored, and let d be the fixed number of bits set for each. Then

$$\phi = (1 - {}^{d}/_{N})^{n}$$

It follows that an item is falsely recognized if all d bits are set to one, which happens with probaility $P = (1 - \phi)^d$.

If we assume $d \ll N$, and we take the log base 2 of ϕ we get approximately

$$\begin{aligned} \log_2 \phi &= \ln(1 - {}^{d}/_{N})^{n} \log_2(e) \\ &= -n({}^{d}/_{N}) \log_2(e). \end{aligned}$$

Therefore

$$N = -n \log_2(P) \frac{\log_2(e)}{\log_2(\phi) \log_2(1 - \phi)}$$

References.

[1] J. R. Bitner. “Heuristics That Dynamically Organize Data Structures”. In: *SIAM J. Comput.* 8.1 (1979), pp. 82–110. DOI: 10.1137/0208007.

[2] B. H. Bloom. “Space/Time Trade-offs in Hash Coding with Allowable Errors”. In: *Commun. ACM* 13.7 (1970), pp. 422–426. DOI: 10.1145/362686.362692.

[3] T.H. Cormen et al. *Introduction to Algorithms*. McGraw-Hill, 2009.

[4] R. L. Rivest. “On Self-Organizing Sequential Search Heuristics”. In: *Commun. ACM* 19.2 (1976), pp. 63–67. DOI: 10.1145/359997.360000.

[5] D. D. Sleator and R. E. Tarjan. “Amortized Efficiency of List Update and Paging Rules”. In: *Commun. ACM* 28.2 (1985), pp. 202–208. DOI: 10.1145/2786.2793.

[6] D. D. Sleator and R. E. Tarjan. “Self-adjusting binary search trees”. In: *J. ACM* 32.3 (1985), pp. 652–686. DOI: 10.1145/3828.3835.

[7] R. E. Tarjan. *Data Structures and Network Algorithms*. Society for Industrial and Applied Mathematics, 1983. DOI: 10.1137/1.9781611970265.